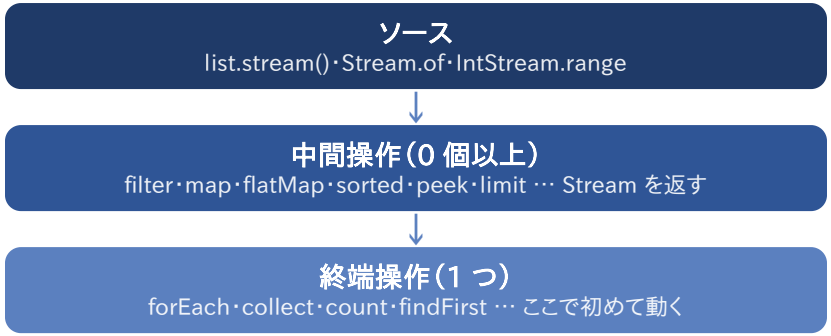
Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 3 Java ストリーム API　#06

ストリームの生成と中間操作

パイプラインと遅延評価・ストリームの作り方・filter／map／flatMap／sorted／peek など

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、Java SE 17 API の java.util.stream パッケージの説明と各メソッド、The Java Tutorials の Aggregate Operations）を書いた。コードの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 ソース → 中間操作 → 終端操作



性質	中身
遅延評価	終端操作がなければ中間操作のラムダ式は呼ばれない
1 回だけ	使用済みの Stream を使うと IllegalStateException
保存しない	元のコレクションは変わらない
短絡	limit・findFirst などで無限ストリームも終わる

> エラー文：stream has already been operated upon or closed

出典：API java.util.stream パッケージの説明「Stream operations and pipelines」／試験内容チェックリスト 3

第1章

ストリームのしくみ

いつ、どの順に動くか。

遅延評価と、要素の流れる順

終端操作なし

```
Stream.of("a","bb","ccc")
  .filter(x -> { print("f"+x); return x.length() > 1; })
  .map(x -> { print("m"+x); return x; });
println("end");
// end だけ（filter も map も動かない）
```

終端操作あり：要素ごとに縦に流れる

```
Stream.of("a","bb","ccc")
  .peek(x -> print("p"+x))
  .filter(x -> x.length() > 1)
  .forEach(x -> print("F"+x));
// pa pbb Fbb pccc Fccc
```

要素	peek	filter	forEach
"a"	pa	落ちる	—
"bb"	pbb	通る	Fbb
"ccc"	pccc	通る	Fccc

> 「全部 peek → 全部 filter」ではなく、1 要素ずつ最後まで流れる。

出典：API java.util.stream パッケージの説明「Laziness」、Stream.peek／JDK 17 での実行結果

短絡操作と、状態を持つ操作

```
Stream.of(1,2,3,4)
  .filter(n -> { print("f"+n); return n % 2 == 0; })
  .map(n -> { print("m"+n); return n; })
  .findFirst();
// f1f2m2 → 3 と 4 は触らない
```

```
Stream.of("b","a","c").sorted()
  .peek(System.out::print).limit(1).forEach(...);
// a!  sorted が全部集めて並べ、先頭だけ流す
Stream.of("b","a","c")
  .peek(System.out::print).limit(1).forEach(...);
// b!
```

分類	中間操作
状態を持たない	filter・map・flatMap・peek
状態を持つ	distinct・sorted・limit・skip
短絡（中間）	limit・takeWhile
短絡（終端）	findFirst・findAny・anyMatch・allMatch・noneMatch

> 無限ストリーム＋sorted は、終端操作を呼ぶと終わらない（組み立てるだけなら何も起きない）。

出典：API java.util.stream パッケージの説明「Stateless behaviors」「Short-circuiting」／JDK 17 での実行結果

使用済みのストリーム

```
Stream<String> s = Stream.of("a","b");
s.forEach(System.out::print);   // ab
s.forEach(System.out::print);   // IllegalStateException
```

```
Stream<String> s = Stream.of("a","b");
s.map(String::toUpperCase);     // 戻り値を捨てた
s.forEach(System.out::print);   // IllegalStateException
```

```
Stream<Integer> t = s.filter(n -> n > 1);
t.count();   // ○
s.count();   // × s は filter した時点で使用済み
```

遅延バインディング

```
List<String> src = new ArrayList<>(List.of("a","b"));
Stream<String> s = src.stream();
src.add("c");
s.count();   // 3（終端操作の開始時点の中身）
```

> 終端操作の**途中**でソースを変えるのは干渉（non-interference 違反）で、ConcurrentModificationException などになりうる。

出典：API java.util.stream パッケージの説明「Non-interference」、Stream／JDK 17 での実行結果

第2章

ストリームの生成

有限・無限、参照型・基本データ型。

ストリームの作り方

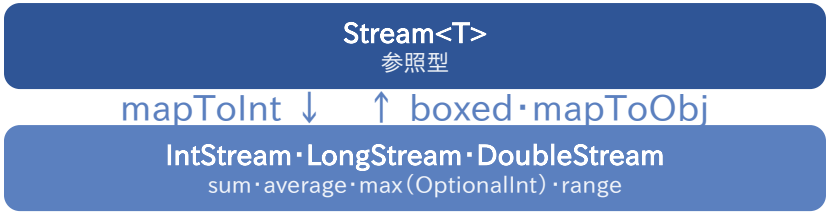
書き方	中身	注意
list.stream()	コレクションの要素	parallelStream は並列（#09）
Stream.of(1, 2, 3)	1, 2, 3	int[] を渡すと要素 1 つの Stream<int[]>
Arrays.stream(arr, 1, 3)	arr[1], arr[2]	int[] なら IntStream、終わりは含まない
IntStream.range(1, 5)／rangeClosed(1, 5)	1〜4／1〜5	range(5, 1) は 0 個
Stream.iterate(1, n -> n * 2)	1, 2, 4, 8 …	無限。limit で止める
Stream.iterate(1, n -> n < 20, n -> n * 3)	1, 3, 9	Java 9〜。for 文と同じ
Stream.generate(() -> "x")	x, x, x …	無限
Stream.empty()・Stream.ofNullable(null)	0 個	Stream.of((String) null) は 1 個
Stream.concat(a, b)・Stream.builder()	a のあと b・add した順	concat は static で引数 2 つ
"abc".chars()	97, 98, 99	IntStream（Stream<Character> ではない）

出典：API Collection.stream・Stream.of・iterate・generate・empty・ofNullable・concat・builder、Arrays.stream、IntStream.range・rangeClosed、CharSequence.chars／JDK 17 での実行結果

配列とストリーム

```
int[] arr = {1, 2, 3, 4, 5};
Arrays.stream(arr, 1, 3).sum()  // 2+3 = 5
Arrays.asList(arr).size()       // 1 (List<int[]>)
Stream.of(arr).count()          // 1 (Stream<int[]>)
IntStream.of(arr).count()       // 5
```

int[] は T... に展開されない。配列 1 つが要素になる。



Stream に mapToObj はない、IntStream の map は int → int。

出典：API Arrays.stream・Arrays.asList・Stream.of・IntStream.of・mapToObj・boxed、Stream.mapToInt／JDK 17 での実行結果

第3章

中間操作

変換・展開・選別・並べ替え。

map と flatMap と mapMulti

map：1 対 1

```
Stream.of("a","bb").map(String::length)  // 1, 2
```

flatMap：1 対多（平らにつなぐ）

```
Stream.of(List.of(1,2), List.of(3), List.<Integer>of())
  .flatMap(List::stream).map(n -> n * 10)
// [10, 20, 30]（空の List からは何も出ない）
Stream.of(1,2).flatMap(n -> Stream.of(n, n))  // [1, 1, 2, 2]
Stream.of("x","yy").flatMapToInt(String::chars).count() // 3
```

mapMulti（Java 16〜）

```
Stream.of(1,2,3).<String>mapMulti((n, sink) -> {
  if (n != 2) { sink.accept(n+"a"); sink.accept(n+"b"); }
})   // 1a1b3a3b
```



出典：API Stream.map・flatMap・flatMapToInt・mapMulti（@since 16）／JDK 17 での実行結果

選別：distinct・limit・skip・takeWhile・dropWhile

```
Stream.of(3,1,3,2,1).distinct()        // 3, 1, 2
  .sorted(Comparator.reverseOrder())    // 3, 2, 1
Stream.of("a","b","c","d","e").skip(1).limit(3)  // bcd
Stream.iterate(1, n -> n + 1).filter(n -> n % 2 == 0).limit(3)
// 2, 4, 6（無限でも limit で終わる）
```

[1, 2, 3, 10, 1, 2] に n < 5

操作	結果
filter	[1, 2, 3, 1, 2]
takeWhile	[1, 2, 3]（10 で止まる）
dropWhile	[10, 1, 2]（10 から後ろ全部）

> takeWhile・dropWhile は Java 9〜。条件が一度 false になったら切り替わる。

出典：API Stream.distinct・skip・limit・takeWhile・dropWhile／JDK 17 での実行結果

sorted と toList

```
Stream.of("aa","b","cc","d")
  .sorted(Comparator.comparing(String::length)
          .thenComparing(Comparator.reverseOrder()))
// [d, b, cc, aa]
```

> 引数なしの sorted は自然順序。Comparable でない要素なら実行時に ClassCastException。

	stream.toList()	collect(Collectors.toList())
導入	Java 16	Java 8
変更	できない（add で UnsupportedOperationException）	仕様上の保証なし（実装は ArrayList）

```
IntStream.of(3,1,2).sorted().boxed()
  .collect(Collectors.toList())   // [1, 2, 3]
```

出典：API Stream.sorted・toList（@since 16）、Collectors.toList、Comparator.comparing・thenComparing／JDK 17 での実行結果

本番での問われ方

ねらい	見るところ
何が表示されるか	終端操作があるか（なければ peek・map は動かない）
出力の順	要素ごとに縦に流れる。sorted は全部集めてから。findFirst・limit で途中で止まる
例外	同じ Stream の 2 回使用は IllegalStateException、toList() に add は UnsupportedOperationException
生成	range と rangeClosed、iterate の 3 引数版、Stream.of(int[])、ofNullable
型	IntStream の map は int → int、Stream に mapToObj はない、boxed
展開	flatMap と map の違い、takeWhile・dropWhile と filter の違い

出典：試験内容チェックリスト 3／本資料の問題演習（全 30 問、JDK 17 で確認）